

PFE Baseline: ResNet-50 on CIFAR-10

A Complete Technical Explanation

Covering:

- Reproducibility & Seeding
- ResNet-50 Architecture & CIFAR-10 Adaptation
- Data Preprocessing & Augmentation
- Training: Loss, Optimizer, Scheduler
- Evaluation Metrics
- Model Complexity Analysis
- Prediction Pipeline

March 15,

2026

Contents

1	Overview	2
2	Reproducibility & Seeding	2
2.1	Why Reproducibility Matters	2
2.2	PRNG and Seed Propagation	2
2.3	cuDNN Determinism	3
2.4	DataLoader Worker Seeding	3
3	The ResNet-50 Architecture	3
3.1	Background: Residual Learning	3
3.2	Bottleneck Block in ResNet-50	4
3.3	ResNet-50 Layer Structure	4
3.4	CIFAR-10 Adaptation	4
3.5	Transfer Learning	5
4	Dataset and Preprocessing	5
4.1	CIFAR-10 Statistics	5
4.2	Normalization	6
4.3	Training Augmentations	6
5	Training	7
5.1	Loss Function: Cross-Entropy with Label Smoothing	7
5.2	Optimizer: SGD with Momentum and Weight Decay	7
5.3	Learning Rate Scheduler: Cosine Annealing	8
5.4	Training Loop	9
5.5	Model Checkpointing	9
6	Evaluation Metrics	9
6.1	Confusion Matrix	9
6.2	Per-Class Precision, Recall, and F1	9
6.3	Macro-Averaged Metrics	10
6.4	Overall Accuracy	10
7	Model Complexity Metrics	11
7.1	Parameter Count	11
7.2	Model Size on Disk	11
7.3	FLOPs (Floating Point Operations)	11
7.4	Inference Latency	11
8	Prediction Pipeline	12
8.1	Preprocessing Consistency	12
8.2	Softmax Probabilities	12
9	Batch Normalization	13
10	Summary Table	14
11	Conclusion	14

1 Overview

This document provides a detailed mathematical and conceptual explanation of a PyTorch implementation that trains a **ResNet-50** model on the **CIFAR-10** dataset. The pipeline covers:

1. Reproducibility configuration
2. Architecture adaptation for small images
3. Data loading and augmentation
4. Training loop with loss, optimizer, and learning rate scheduling
5. Evaluation metrics
6. Model complexity measurement
7. Prediction on custom images

The design goal is to serve as a *clean baseline* before applying model compression techniques (pruning, quantization, distillation, LoRA).

2 Reproducibility & Seeding

2.1 Why Reproducibility Matters

Deep learning experiments involve multiple sources of randomness:

- Random weight initialization
- Stochastic gradient descent mini-batch sampling
- Data augmentation (random flips, crops)
- cuDNN’s non-deterministic convolution algorithms
- OS-level thread scheduling in DataLoaders

Setting a fixed **seed** s ensures that any pseudo-random number generator (PRNG) produces the same sequence of numbers across runs.

2.2 PRNG and Seed Propagation

Most PRNGs (e.g., Mersenne Twister used by Python’s **random** and NumPy) operate as a deterministic function:

$$x_{n+1} = f(x_n), \quad x_0 = s$$

where s is the seed and f is a transition function designed to have a very long period.

The code seeds **four separate PRNGs**:

Seeding Strategy

$$\text{PRNG}_{\text{Python}} \leftarrow s, \quad \text{PRNG}_{\text{NumPy}} \leftarrow s, \quad \text{PRNG}_{\text{CPU}} \leftarrow s, \quad \text{PRNG}_{\text{GPU}} \leftarrow s$$

with $s = 42$.

```

1 SEED = 42
2 random.seed(SEED)
3 np.random.seed(SEED)
4 torch.manual_seed(SEED)
5 torch.cuda.manual_seed(SEED)
6 torch.cuda.manual_seed_all(SEED)
7
8 torch.backends.cudnn.deterministic = True
9 torch.backends.cudnn.benchmark     = False

```

Listing 1: Reproducibility setup

2.3 cuDNN Determinism

Setting `cudnn.deterministic = True` forces cuDNN to use deterministic convolution algorithms, which may be slower but guarantee identical floating-point results across runs. Setting `benchmark = False` disables the automatic algorithm selection that cuDNN uses to optimize performance (which can introduce non-determinism).

2.4 DataLoader Worker Seeding

PyTorch DataLoaders use multiple worker processes. Each worker inherits the base seed and gets an offset:

$$s_{\text{worker}} = (\text{torch.initial_seed}()) \bmod 2^{32}$$

```

1 def seed_worker(worker_id):
2     worker_seed = torch.initial_seed() % 2**32
3     np.random.seed(worker_seed)
4     random.seed(worker_seed)

```

Listing 2: Worker seed function

A `torch.Generator` with the fixed seed is also passed to the `DataLoader` to control the shuffling order of training samples.

3 The ResNet-50 Architecture

3.1 Background: Residual Learning

ResNets [1] were introduced to solve the **vanishing gradient problem** in very deep networks. The core idea is the *residual (skip) connection*:

Residual Block

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

where $\mathbf{x}$ is the input, $\mathcal{F}$ is the residual function (a few conv layers), and $\mathbf{x}$ is the identity shortcut.

The motivation: if the optimal function is close to the identity, it is easier for the network to learn $\mathcal{F}(\mathbf{x}) \approx \mathbf{0}$ than to learn $\mathcal{F}(\mathbf{x}) \approx \mathbf{x}$ directly.

3.2 Bottleneck Block in ResNet-50

ResNet-50 uses **bottleneck** blocks (3 layers each) instead of the basic 2-layer block in ResNet-18/34:

$$\text{Block: } \underbrace{1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1}_{\text{reduce} \rightarrow \text{process} \rightarrow \text{expand}}$$

Concretely, for a block with c output channels:

$$\begin{aligned} \mathbf{z}_1 &= \text{BN}(\text{ReLU}(W_1^{1 \times 1} * \mathbf{x})) \\ \mathbf{z}_2 &= \text{BN}(\text{ReLU}(W_2^{3 \times 3} * \mathbf{z}_1)) \\ \mathbf{z}_3 &= \text{BN}(W_3^{1 \times 1} * \mathbf{z}_2) \\ \mathbf{y} &= \text{ReLU}(\mathbf{z}_3 + \mathbf{x}) \end{aligned}$$

When dimensions change, a 1×1 projection shortcut is used:

$$\mathbf{y} = \text{ReLU}(\mathcal{F}(\mathbf{x}) + W_s \mathbf{x})$$

3.3 ResNet-50 Layer Structure

The full architecture has 5 stages:

Table 1: ResNet-50 Architecture Stages

Stage	Output Size	Block	# Blocks	Channels
Conv1	32×32	Conv 3×3 , stride 1	1	64
Layer 1	32×32	Bottleneck	3	64, 256
Layer 2	16×16	Bottleneck	4	128, 512
Layer 3	8×8	Bottleneck	6	256, 1024
Layer 4	4×4	Bottleneck	3	512, 2048
AvgPool	1×1	Global Avg Pool	—	2048
FC	10	Linear	—	10

3.4 CIFAR-10 Adaptation

Problem: The original ResNet-50 was designed for ImageNet (224×224 images). Its first layer is a 7×7 convolution with stride 2, followed by a max-pooling layer:

$$224 \xrightarrow{\text{conv}, s=2} 112 \xrightarrow{\text{maxpool}, s=2} 56$$

Applied to CIFAR-10's 32×32 images:

$$32 \xrightarrow{\text{conv}, s=2} 16 \xrightarrow{\text{maxpool}, s=2} 8$$

This aggressive downsampling destroys most spatial information before the network has even started learning features.

Solution: Replace the first convolution and remove max-pooling:

```

1 def build_model(num_classes=10):
2     model = models.resnet50(pretrained=True)
3     # 7x7 stride-2 -> 3x3 stride-1: no spatial downsampling
4     model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,
5                             padding=1, bias=False)
6     # Remove maxpool: prevents 32->16 collapse
7     model.maxpool = nn.Identity()
8     model.fc = nn.Linear(model.fc.in_features, num_classes)
9     return model

```

Listing 3: Architecture adaptation for 32x32 images

Spatial Flow After Adaptation

$$32 \xrightarrow{\text{Conv } 3 \times 3, s=1} 32 \xrightarrow{\text{Identity}} 32 \xrightarrow{\text{Layer1}} 32 \xrightarrow{\text{Layer2}} 16 \xrightarrow{\text{Layer3}} 8 \xrightarrow{\text{Layer4}} 4 \xrightarrow{\text{GAP}} 1 \xrightarrow{\text{FC}} 10$$

The `nn.Identity()` replaces max-pooling with a no-op, preserving the full 32×32 spatial resolution into the first residual stack.

3.5 Transfer Learning

The model loads **ImageNet pretrained weights** (`pretrained=True`). Only the first conv and the final FC layer are replaced (and thus randomly re-initialized). The residual stages retain ImageNet features, allowing faster convergence on CIFAR-10 via transfer learning.

4 Dataset and Preprocessing

4.1 CIFAR-10 Statistics

CIFAR-10 contains 60,000 color images of size 32×32 in 10 classes (50,000 train, 10,000 test). The dataset statistics used for normalization:

CIFAR-10 Channel Statistics

$$\mu = (0.4914, 0.4822, 0.4465), \quad \sigma = (0.2023, 0.1994, 0.2010)$$

per channel (R, G, B), computed over the training set.

4.2 Normalization

Each input image $\mathbf{x} \in [0, 1]^{3 \times 32 \times 32}$ (after `ToTensor()`) is normalized channel-wise:

$$\hat{x}_{c,h,w} = \frac{x_{c,h,w} - \mu_c}{\sigma_c}, \quad c \in \{R, G, B\}$$

This ensures each channel has approximately zero mean and unit variance, which stabilizes training.

4.3 Training Augmentations

Three augmentations are applied *only during training* (not at test time):

Table 2: Training Data Augmentations

Augmentation	Parameters	Purpose
RandomHorizontalFlip	$p = 0.5$	Left-right symmetry
RandomCrop	32×32 , padding = 4	Translation invariance
ColorJitter	brightness, contrast, saturation = 0.2	Color invariance

Random Crop with Padding The image is padded with 4 pixels on each side (to 40×40) and a random 32×32 patch is cropped. This simulates small translations:

$$\tilde{\mathbf{x}} = \text{Crop}_{32 \times 32}(\text{Pad}_4(\mathbf{x}))$$

ColorJitter Randomly adjusts brightness b , contrast c , and saturation s within $[1 - \delta, 1 + \delta]$ for $\delta = 0.2$:

$$\tilde{x}_{\text{channel}} = \text{clip}(\alpha \cdot x_{\text{channel}} + \beta, 0, 1)$$

```

1 transform_train = transforms.Compose([
2     transforms.RandomHorizontalFlip(),
3     transforms.RandomCrop(32, padding=4),
4     transforms.ColorJitter(brightness=0.2, contrast=0.2,
5     saturation=0.2),
6     transforms.ToTensor(),
7     transforms.Normalize(CIFAR_MEAN, CIFAR_STD)
8 ])
9 transform_test = transforms.Compose([
10    transforms.ToTensor(),
11    transforms.Normalize(CIFAR_MEAN, CIFAR_STD)
12 ])

```

Listing 4: Data transforms

Key Rule

The test transform is **identical to training** in normalization but has **no augmentation**. Using the same mean/std is critical — any mismatch creates a distribution shift between training and inference.

5 Training

5.1 Loss Function: Cross-Entropy with Label Smoothing

The standard cross-entropy loss for a sample with true class y and predicted logits $\mathbf{z} \in \mathbb{R}^K$ is:

$$\mathcal{L}_{\text{CE}}(y, \mathbf{z}) = -\log \frac{e^{z_y}}{\sum_{k=1}^K e^{z_k}}$$

Label Smoothing ($\epsilon = 0.1$) replaces the hard one-hot target $q(k) = \mathbb{1}[k = y]$ with a soft distribution:

Label Smoothing

$$q_{\text{smooth}}(k) = (1 - \epsilon) \mathbb{1}[k = y] + \frac{\epsilon}{K}$$

$$\mathcal{L}_{\text{LS}} = -\sum_{k=1}^K q_{\text{smooth}}(k) \log p(k)$$

where $p(k) = \text{softmax}(\mathbf{z})_k$ and $K = 10$.

With $\epsilon = 0.1$ and $K = 10$:

$$q_{\text{smooth}}(k) = \begin{cases} 0.9 + 0.01 = 0.91 & k = y \\ 0.01 & k \neq y \end{cases}$$

Label smoothing reduces *overconfidence*: the model is penalized if it assigns probability 1 to the correct class, which improves generalization and calibration.

```
1 criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
```

Listing 5: Loss function

5.2 Optimizer: SGD with Momentum and Weight Decay

Stochastic Gradient Descent with momentum and weight decay (L2 regularization):

SGD with Momentum

$$\mathbf{v}_{t+1} = \gamma \mathbf{v}_t - \eta (\nabla_{\theta} \mathcal{L}(\theta_t) + \lambda \theta_t) \quad (1)$$

$$\theta_{t+1} = \theta_t + \mathbf{v}_{t+1} \quad (2)$$

where:

- $\eta = 0.1$ — learning rate
- $\gamma = 0.9$ — momentum coefficient
- $\lambda = 5 \times 10^{-4}$ — weight decay (L2 regularization strength)
- $\mathbf{v}_t$ — velocity (exponential moving average of gradients)

Why SGD over Adam for CIFAR? SGD with momentum consistently outperforms Adam on CIFAR-10 at convergence because Adam’s adaptive learning rates can lead to sharp minima that generalize less well. SGD finds flatter minima with better test accuracy when tuned properly.

Weight Decay as L2 Regularization Adding $\lambda \|\theta\|^2/2$ to the loss is equivalent to multiplying weights by $(1 - \eta\lambda)$ at each step, preventing weights from growing too large.

```
1 optimizer = torch.optim.SGD(model.parameters(), lr=0.1,
2                               momentum=0.9, weight_decay=5e-4)
```

Listing 6: Optimizer

5.3 Learning Rate Scheduler: Cosine Annealing

The learning rate follows a cosine decay schedule over $T_{\max}$ epochs:

Cosine Annealing LR

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T_{\max}}\right) \right)$$

With $\eta_{\min} = 0$ (default), $\eta_{\max} = 0.1$, $T_{\max} = 40$:

$$\eta_t = 0.05 \left(1 + \cos\left(\frac{\pi t}{40}\right) \right)$$

- At $t = 0$: $\eta_0 = 0.1$ (warm start)
- At $t = 20$: $\eta_{20} = 0.05$ (midpoint)
- At $t = 40$: $\eta_{40} = 0$ (cold end)

This smooth decay avoids sudden changes in the optimization landscape and typically outperforms step-decay schedules by allowing the optimizer to settle into a minimum more gracefully.

```
1 scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
2     optimizer, T_max=EPOCHS)
```

Listing 7: Scheduler

5.4 Training Loop

Each epoch performs one full pass over the training set:

```

1 def train_epoch(model, loader, optimizer, criterion):
2     model.train()
3     total_loss, correct, total = 0, 0, 0
4     for inputs, labels in loader:
5         inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
6         optimizer.zero_grad()
7         outputs = model(inputs)
8         loss = criterion(outputs, labels)
9         loss.backward()
10        optimizer.step()
11        total_loss += loss.item()
12        correct += outputs.argmax(1).eq(labels).sum().item()
13        total += labels.size(0)
14    return total_loss / len(loader), correct / total

```

Listing 8: Single training epoch

The forward-backward pass for one mini-batch of size B :

1. **Forward pass:** $\hat{\mathbf{y}} = f_{\theta}(\mathbf{X}) \in \mathbb{R}^{B \times K}$
2. **Loss:** $\mathcal{L} = \frac{1}{B} \sum_{i=1}^B \mathcal{L}_{\text{LS}}(y_i, \hat{y}_i)$
3. **Backward pass:** $\nabla_{\theta} \mathcal{L}$ via backpropagation (chain rule)
4. **Update:** $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$

5.5 Model Checkpointing

The best model (highest validation accuracy) is saved:

$$\hat{\theta}^* = \underset{\hat{\theta}}{\text{Acc}}(f_{\hat{\theta}}, \mathcal{D}_{\text{val}})$$

This prevents returning a model that overfit the last few epochs.

6 Evaluation Metrics

6.1 Confusion Matrix

For a K -class problem, the confusion matrix $\mathbf{C} \in \mathbb{Z}^{K \times K}$ is:

$$C_{ij} = \#\{\text{samples truly class } i \text{ predicted as class } j\}$$

The diagonal C_{ii} represents correct predictions for class i .

6.2 Per-Class Precision, Recall, and F1

For each class k :

Classification Metrics

$$\text{Precision}_k = \frac{C_{kk}}{\sum_{i=1}^K C_{ik}} = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k} \quad (3)$$

$$\text{Recall}_k = \frac{C_{kk}}{\sum_{j=1}^K C_{kj}} = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k} \quad (4)$$

$$\text{F1}_k = 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \quad (5)$$

6.3 Macro-Averaged Metrics

Macro-averaging computes the metric for each class independently and takes the un-weighted mean:

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Precision}_k$$

$$\text{Recall}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Recall}_k$$

$$\text{F1}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{F1}_k$$

Macro averaging treats all classes equally regardless of sample count, which is appropriate for the balanced CIFAR-10 test set (1,000 samples per class).

6.4 Overall Accuracy

$$\text{Accuracy} = \frac{\sum_{k=1}^K C_{kk}}{\sum_{i,j} C_{ij}} = \frac{\text{Number of correct predictions}}{\text{Total predictions}}$$

```

1 model.load_state_dict(torch.load(SAVE_PATH, map_location=DEVICE))
2 model.eval()
3 all_preds, all_labels = [], []
4 with torch.no_grad():
5     for inputs, labels in test_loader:
6         preds = model(inputs.to(DEVICE)).argmax(1).cpu()
7         all_preds.extend(preds.numpy())
8         all_labels.extend(labels.numpy())

```

Listing 9: Full evaluation pass

`model.eval()` and `torch.no_grad()`

`model.eval()` switches BatchNorm to use running statistics (not batch statistics) and disables Dropout. `torch.no_grad()` disables gradient tracking, reducing mem-

ory usage by $\sim 50\%$ and speeding up inference.

7 Model Complexity Metrics

7.1 Parameter Count

The total number of learnable parameters:

$$P = \sum_{\ell=1}^L |\theta^{(\ell)}|$$

where $|\theta^{(\ell)}|$ is the number of scalar parameters in layer ℓ .

For a convolutional layer with C_{out} filters of size $k \times k$ applied to C_{in} channels:

$$P_{\text{conv}} = C_{\text{out}} \times C_{\text{in}} \times k \times k + C_{\text{out}} \cdot \mathbb{K}[\text{bias}]$$

ResNet-50 has approximately **25.6 million parameters**.

7.2 Model Size on Disk

Each float32 parameter uses 4 bytes:

$$\text{Size} = P \times 4 \text{ bytes} \approx P \times 4 \times 10^{-6} \text{ MB}$$

7.3 FLOPs (Floating Point Operations)

FLOPs count the computational work in a forward pass. For a single convolutional layer:

$$\text{FLOPs}_{\text{conv}} = 2 \times C_{\text{in}} \times C_{\text{out}} \times k^2 \times H_{\text{out}} \times W_{\text{out}}$$

The factor of 2 accounts for multiply-accumulate operations (one multiply + one add).

GFLOPs = FLOPs/ 10^9 .

The code uses `thop.profile` to count FLOPs automatically:

```

1 dummy = torch.randn(1, 3, 32, 32).to(DEVICE)
2 flops, _ = profile(model, inputs=(dummy,), verbose=False)
3 print(f"FLOPs: {flops/1e9:.3f} GFLOPs")

```

Listing 10: FLOPs counting

7.4 Inference Latency

Measured over 500 runs after 10 warm-up passes (to amortize GPU kernel launch overhead and JIT compilation):

$$t_{\text{inf}} = \frac{1}{N} \sum_{i=1}^N \left(t_{\text{end}}^{(i)} - t_{\text{start}}^{(i)} \right), \quad N = 500$$

```

1 with torch.no_grad():
2     for _ in range(10): model(dummy)    # warm-up
3     times = []
4     for _ in range(500):
5         t0 = time.time()
6         model(dummy)
7         times.append(time.time() - t0)
8 inf_ms = np.mean(times) * 1000

```

Listing 11: Latency measurement

Important

For rigorous GPU benchmarking, one should use `torch.cuda.synchronize()` before timing, as CUDA operations are asynchronous. The current code measures wall-clock time which may be less accurate on GPU.

8 Prediction Pipeline

8.1 Preprocessing Consistency

The prediction transform must exactly match the training transform (excluding augmentations):

```

1 predict_transform = transforms.Compose([
2     transforms.Resize((32, 32)),
3     transforms.ToTensor(),
4     transforms.Normalize(CIFAR_MEAN, CIFAR_STD)
5 ])

```

Listing 12: Prediction transform

Distribution Shift Warning

Using different normalization statistics at inference time would shift the input distribution away from what the model was trained on. Even a small mismatch (e.g., wrong mean) degrades accuracy significantly because the model has learned features specific to the normalized distribution.

8.2 Softmax Probabilities

Raw model output $\mathbf{z} \in \mathbb{R}^K$ (logits) is converted to a probability distribution:

Softmax

$$p(k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k \in \{1, \dots, K\}$$

Properties: $p(k) \in (0, 1)$ and $\sum_k p(k) = 1$.

The predicted class:

$$\hat{y} =_k p(k | \mathbf{x})$$

The confidence:

$$\text{conf} = \max_k p(k | \mathbf{x})$$

```

1 def predict_image(image_path, model, show=True):
2     img      = Image.open(image_path).convert('RGB')
3     tensor   = predict_transform(img).unsqueeze(0).to(DEVICE)
4     model.eval()
5     with torch.no_grad():
6         probs = torch.softmax(model(tensor), dim=1).squeeze().cpu()
7         pred_idx = probs.argmax()
8         confidence = probs[pred_idx]
```

Listing 13: Single image prediction

`unsqueeze(0)` adds the batch dimension: $\text{shape } [3, 32, 32] \rightarrow [1, 3, 32, 32]$.

9 Batch Normalization

Throughout ResNet-50, Batch Normalization (BN) is applied after every convolution:

Batch Normalization

Given a mini-batch $\mathcal{B} = \{x_1, \dots, x_B\}$:

$$\mu_{\mathcal{B}} = \frac{1}{B} \sum_{i=1}^B x_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{B} \sum_{i=1}^B (x_i - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

where γ, β are learnable parameters and $\epsilon \approx 10^{-5}$.

During **training**: statistics are computed per-batch.

During **inference**: running statistics (exponential moving averages accumulated during training) are used:

$$\mu_{\text{run}} \leftarrow (1 - \alpha) \mu_{\text{run}} + \alpha \mu_{\mathcal{B}}, \quad \alpha = 0.1$$

This is why `model.eval()` must be called at test time.

10 Summary Table

Table 3: Summary of Key Hyperparameters and Design Choices

Component	Choice	Key Equation / Value
Architecture	ResNet-50 (adapted)	Bottleneck, skip connections
Input conv	3×3 , stride 1	No spatial downsampling
Max-pool	Removed (Identity)	Preserves 32×32 resolution
Pretrained	ImageNet weights	Transfer learning
Epochs	40	—
Batch size	128	—
Loss	CE + Label Smoothing	$\epsilon = 0.1$
Optimizer	SGD + Momentum	$\eta = 0.1, \gamma = 0.9, \lambda = 5 \times 10^{-4}$
LR Schedule	Cosine Annealing	$\eta_t = 0.05(1 + \cos(\pi t/40))$
Normalization	CIFAR-10 stats	$\mu = (0.491, 0.482, 0.446)$
Augmentations	Flip, Crop, Jitter	Train only
Metric	Macro F1, Accuracy	Balanced 10-class evaluation

11 Conclusion

This baseline implements a carefully adapted ResNet-50 for CIFAR-10 with several important design decisions:

1. **Architecture adaptation:** Replacing the 7×7 stride-2 convolution with a 3×3 stride-1 convolution, and removing the max-pool layer, prevents the aggressive spatial downsampling that would destroy the small 32×32 spatial resolution.
2. **Label smoothing:** Prevents overconfidence and improves calibration at the cost of a slightly lower raw training accuracy.
3. **SGD + Cosine Annealing:** A well-established combination that produces flatter minima and better generalization than Adam on CIFAR-10.
4. **Preprocessing consistency:** The same normalization statistics are used in training, evaluation, and prediction — any mismatch would degrade performance.
5. **Full reproducibility:** All random sources are seeded, including DataLoader workers, ensuring experiments are repeatable.

This baseline is designed to be the reference point for compression experiments: any pruned, quantized, or distilled model should be compared against these metrics (accuracy, FLOPs, size, latency).

References

- [1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. CVPR 2016.

- [2] Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.
- [3] Ioffe, S., & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. ICML 2015.
- [4] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). *Rethinking the Inception Architecture for Computer Vision*. CVPR 2016. (Label Smoothing introduced here.)
- [5] Loshchilov, I., & Hutter, F. (2017). *SGDR: Stochastic Gradient Descent with Warm Restarts*. ICLR 2017.